

Panduan Workflow, Timeline, dan Test Postman

Project Reservasi Ruang - satu branch main - fokus minim konflik

1. Prinsip kerja wajib

Satu branch main boleh dipakai, tetapi syaratnya disiplin. Tidak ada workflow yang benar-benar 100% anti konflik. Target dokumen ini adalah membuat kerja tim paling aman untuk pemula: urutan jelas, file owner jelas, pull sebelum edit, push kecil, dan test sebelum sync.

- Sebelum edit: klik Source Control di VS Code, lalu Sync Changes atau Pull.
- Sebelum menyentuh file: tulis di grup, contoh: Saya lock Room.java dan RoomService.java.
- Edit hanya file sesuai jobdesk. Jangan format seluruh project.
- Commit kecil. Satu commit hanya satu modul kecil yang selesai.
- Sebelum push: jalankan test. Minimal pastikan aplikasi masih start.
- Jika konflik: stop, jangan asal accept. Hubungi pemilik file.
- Koordinator berhak freeze push jika main rusak.

2. Pembagian jobdesk

Anggota	Modul	Fokus
Ajda Mutiara Zahra	NotificationService, Notification	Notifikasi, status baca, penerima, waktu kirim.
Amelia Sofiana Makharomi	User, Mahasiswa	Login, logout, ubah profil, ajukan/batalan/status reservasi mahasiswa.
Tadzkiroh Aziziyah Haqia	Satpam, AccessRecord, AccessControlService	Check-in, check-out, kendala akses ruang.
Ali Abdul Fattah 'Alim Kautsar	ReservationService, Room	Data ruang, availability, create/cancel/history reservasi.
Atha Muyassar	Admin, Notifiable	Verifikasi reservasi, setuju, tolak, revisi, penerima notifikasi.
'Arif Rahmaan Jalaluddin Faqiih	Reservation, Approval, ApprovalService	Entity reservasi, status, validasi waktu, approval flow.

3. Workflow GitHub dan VS Code GUI

Setup awal: Koordinator melakukan publish repo dari VS Code.

- Buka project di VS Code.
- Klik ikon Source Control di sidebar kiri.
- Jika belum Git: klik Initialize Repository.
- Stage semua file dengan tombol plus.
- Isi message commit: checkpoint-1-starter.
- Klik Commit.
- Klik Publish Branch atau Publish to GitHub.
- Buka GitHub, masuk Settings - Collaborators, invite 5 anggota.

Clone: Anggota clone dari VS Code dan buka workspace.

- Accept invite GitHub.
- Buka VS Code.
- Klik Source Control atau Welcome screen - Clone Git Repository.
- Paste URL repo GitHub.
- Pilih folder lokal.
- Klik Open dan Trust Workspace.
- Jalankan project, lalu login dengan akun starter.

Pull-edit-test-push: Loop harian yang wajib diikuti.

- Klik Source Control - menu tiga titik - Pull atau Sync Changes.

- Tulis file yang akan diedit di grup.
- Edit file sendiri.
- Jalankan test dari Maven panel atau terminal: mvnw test.
- Stage hanya file sendiri.
- Commit dengan pesan jelas, contoh: Ali - tambah entity Room.
- Klik Sync Changes untuk push.

4. Timeline anti berantakan

1. Koordinator

Apa dilakukan: push starter checkpoint ke GitHub dari kondisi project yang sudah rapi

Kenapa ini dulu: agar semua anggota mulai dari base code yang sama

Diharapkan: repo GitHub berisi starter login, koneksi database, dan template file komentar

Hasil step: 1 sumber utama kerja tim tersedia di branch main

2. Koordinator

Apa dilakukan: invite 5 anggota sebagai collaborator di GitHub

Kenapa ini dulu: agar semua bisa pull dan push ke repo yang sama

Diharapkan: semua anggota mendapat undangan GitHub

Hasil step: anggota bisa accept invite dan clone project

3. Semua anggota

Apa dilakukan: accept invite, clone repo lewat VS Code, lalu buka folder project

Kenapa ini dulu: agar setiap laptop punya copy project yang sama

Diharapkan: project bisa dibuka di VS Code masing-masing

Hasil step: semua siap melakukan pull, edit, commit, dan sync dari Source Control

4. Semua anggota

Apa dilakukan: jalankan project dan login dengan mhs/mhs, adm/adm, stm/stm

Kenapa ini dulu: agar terbukti starter tidak rusak sebelum fitur dibuat

Diharapkan: halaman role bisa dibuka sesuai akun

Hasil step: baseline aplikasi dinyatakan aman untuk mulai development

5. Koordinator

Apa dilakukan: membuat aturan file owner di grup chat

Kenapa ini dulu: karena satu branch main rawan tabrakan jika dua orang edit file yang sama

Diharapkan: setiap anggota tahu file mana yang boleh dia isi

Hasil step: konflik Git turun karena file dikerjakan sesuai jobdesk

6. Amelia

Apa dilakukan: isi User, Mahasiswa, DTO profil, UserService, UserRepository

Kenapa ini dulu: karena Reservation bergantung pada Mahasiswa dan data User

Diharapkan: domain user dasar tersedia dan tidak membocorkan passwordHash

Hasil step: anggota lain bisa memakai Mahasiswa sebagai relasi

7. Ali

Apa dilakukan: isi Room, RoomRepository, RoomService, DTO room, dan halaman daftar room

Kenapa ini dulu: karena reservasi tidak bisa dibuat tanpa data ruangan

Diharapkan: roomId, namaRuangan, gedung, kapasitas, statusAktif siap dipakai

Hasil step: modul ruangan menjadi dasar availability reservasi

8. 'Aarif + Ali

Apa dilakukan: isi ReservationStatus, Reservation, ReservationRepository, ReservationService availability

Kenapa ini dulu: karena alur utama aplikasi adalah reservasi ruang

Diharapkan: reservasi bisa dibuat, divalidasi waktunya, dan dicek bentrok jadwal

Hasil step: mahasiswa punya alur pengajuan reservasi yang bisa dites

9. 'Aarif

Apa dilakukan: isi ApprovalDecision, Approval, ApprovalRepository, ApprovalService

Kenapa ini dulu: karena status reservasi harus diputuskan oleh admin

Diharapkan: approve, reject, dan revision punya data dan aturan yang jelas

Hasil step: reservation status bisa berubah dari hasil approval

10. Atha

Apa dilakukan: isi Admin, Notifiable, AdminController, AdminRestController, dan template approval

Kenapa ini dulu: karena admin adalah aktor yang memakai ApprovalService

Diharapkan: admin bisa melihat reservasi pending dan memberi keputusan

Hasil step: alur approval siap dites dari Postman dan halaman admin

11. Tadzkiroh

Apa dilakukan: isi Satpam, AccessRecord, AccessControlService, AccessRestController, dan template akses

Kenapa ini dulu: karena akses ruang baru valid setelah reservasi disetujui

Diharapkan: satpam bisa mencatat check-in, check-out, dan kendala

Hasil step: riwayat akses ruang mulai terbentuk

12. Ajda

Apa dilakukan: isi Notification, NotificationService, NotificationRepository, DTO, controller, dan template notifikasi

Kenapa ini dulu: karena notifikasi butuh penerima final dari Mahasiswa/Admin/Satpam

Diharapkan: notifikasi bisa dibuat saat status reservasi berubah

Hasil step: pengguna bisa melihat dan menandai notifikasi dibaca

13. Amelia + Ali

Apa dilakukan: hubungkan halaman mahasiswa dengan form reservasi dan list status

Kenapa ini dulu: karena user flow mahasiswa harus utuh setelah ReservationService siap

Diharapkan: mahasiswa bisa mengajukan, membatalkan, dan melihat status reservasi

Hasil step: fitur mahasiswa siap demo awal

14. Atha + 'Aarif

Apa dilakukan: hubungkan halaman admin dengan approval list dan action keputusan

Kenapa ini dulu: karena admin membutuhkan ApprovalService yang sudah stabil

Diharapkan: admin bisa setuju, tolak, atau minta revisi dari UI

Hasil step: fitur approval siap demo awal

15. Tadzkiroh

Apa dilakukan: hubungkan halaman satpam dengan check-in, check-out, dan kendala

Kenapa ini dulu: karena fitur satpam harus memakai reservasi yang sudah approved

Diharapkan: satpam hanya memproses reservasi yang valid

Hasil step: fitur akses ruang siap demo awal

16. Ajda + semua anggota

Apa dilakukan: hubungkan notifikasi ke event reservasi, approval, dan akses

Kenapa ini dulu: karena notifikasi adalah fitur lintas modul

Diharapkan: setiap perubahan penting mengirim Notification

Hasil step: pengguna mendapat informasi status terbaru

17. Semua anggota

Apa dilakukan: rapikan layout, sidebar, error handling, dan validasi input

Kenapa ini dulu: karena fitur sudah saling terhubung dan perlu pengalaman pakai yang rapi

Diharapkan: UI konsisten, error mudah dipahami, dan role tidak bocor

Hasil step: aplikasi siap diuji end-to-end

18. Semua anggota

Apa dilakukan: uji API dengan Postman sesuai urutan: login, room, reservation, approval, access, notification

Kenapa ini dulu: karena bug integrasi paling mudah terlihat dari API

Diharapkan: setiap endpoint punya status code dan response yang benar

Hasil step: daftar bug final bisa dibagi sesuai pemilik modul

19. Koordinator + semua anggota

Apa dilakukan: freeze fitur, pull terakhir, test ulang, lalu siapkan demo

Kenapa ini dulu: karena perubahan besar di akhir rawan merusak main

Diharapkan: tidak ada push fitur baru menjelang presentasi

Hasil step: project stabil untuk demo dan penilaian

5. Urutan test Postman

Endpoint API di bawah adalah target implementasi. Jika endpoint belum dibuat, 404 itu normal. Setelah anggota mengisi controller masing-masing, endpoint ini menjadi checklist test.

Environment Postman: baseUrl=http://localhost:8080, csrfToken kosong dulu, roomId/reservationId/notificationId diisi dari response.

Cara login: GET /login untuk ambil CSRF, lalu POST /login dengan username, password, dan _csrf. Gunakan session cookie yang sama.

Postman Tests script untuk GET /login

```
const html = pm.response.text();
const match = html.match(/name="_csrf" value="([^\"]+)"/);
if (match) {
  pm.environment.set("csrfToken", match[1]);
}
```

Starter login yang sudah ada

Method	Endpoint	Role	Body	Expected	Jika gagal
GET	{{baseUrl}}/login	Semua	Tidak ada body	200 OK, HTML form login	Jika 404: aplikasi belum jalan atau port salah.
POST	{{baseUrl}}/login	Semua	x-www-form-urlencoded: username=mhs, password=mhs, _csrf={{csrfToken}}	302 ke /dashboard atau 200 setelah redirect	Jika 403: ambil csrf baru dari GET /login.
GET	{{baseUrl}}/mahasiswa/dash board	MAHASISWA	Tidak ada body	200 OK, teks ini halaman mahasiswa	Jika 403: login bukan mhs.
GET	{{baseUrl}}/admin/dashboard	ADMIN	Tidak ada body	200 OK, teks ini halaman admin	Jika 403: login bukan adm.
GET	{{baseUrl}}/satpam/dashboar d	SATPAM	Tidak ada body	200 OK, teks ini halaman satpam	Jika 403: login bukan stm.

Room API - dikerjakan Ali

Method	Endpoint	Role	Body	Expected	Jika gagal
POST	{{baseUrl}}/api/rooms	ADMIN	{"namaRuang":"Ruang A101","gedung":"Gedung A","kapasitas":40,"statusAktif":true}	201 Created, mengembalikan roomId	400 jika kapasitas kosong/negatif. 403 jika bukan admin.
GET	{{baseUrl}}/api/rooms	MAHASISWA /ADMIN	Tidak ada body	200 OK, list ruangan	Jika kosong: cek seed/data room.
GET	{{baseUrl}}/api/rooms/active	MAHASISWA /ADMIN	Tidak ada body	200 OK, hanya statusAktif=true	Jika room nonaktif muncul: query salah.
PATCH	{{baseUrl}}/api/rooms/{roomId}/status	ADMIN	{"statusAktif":false}	200 OK, statusAktif berubah	409 jika ruangan sedang dipakai dan aturan final melarang nonaktif.

Reservation API - dikerjakan Ali + 'Aarif

Method	Endpoint	Role	Body	Expected	Jika gagal
POST	{{baseUrl}}/api/reservations/availability	MAHASISWA	{"roomId":"ROOM-001","tanggal":"2026-06-10","jamMulai":"18:00","jamSelesai":"20:00"}	200 OK, available=true/false	400 jika jam salah. 404 jika roomId tidak ada.
POST	{{baseUrl}}/api/reservations	MAHASISWA	{"roomId":"ROOM-001","tanggal":"2026-06-10","jamMulai":"18:00","jamSelesai":"20:00","tujuan":"Rapat UKM"}	201 Created, status awal misalnya PENDING	409 jika jadwal bentrok. 400 jika tujuan kosong.
GET	{{baseUrl}}/api/reservations/me	MAHASISWA	Tidak ada body	200 OK, list reservasi milik mahasiswa login	Jika data user lain muncul: bug authorization.
POST	{{baseUrl}}/api/reservations/{reservationId}/cancel	MAHASISWA	{"alasanPembatalan":"Jadwal berubah"}	200 OK, cancelledAt terisi	409 jika status tidak bisa dibatalkan.

Approval API - dikerjakan Atha + 'Aarif

Method	Endpoint	Role	Body	Expected	Jika gagal
GET	{{baseUrl}}/api/approvals/pending	ADMIN	Tidak ada body	200 OK, list reservasi pending	Jika 403: login bukan admin.
POST	{{baseUrl}}/api/approvals/{reservationId}/approve	ADMIN	{"catatan":"Disetujui"}	200 OK, keputusan setuju, reviewedAt terisi	409 jika reservation bukan status pending.
POST	{{baseUrl}}/api/approvals/{reservationId}/reject	ADMIN	{"catatan":"Ruangan dipakai acara kampus"}	200 OK, keputusan tolak	400 jika catatan wajib tapi kosong.
POST	{{baseUrl}}/api/approvals/{reservationId}/revision	ADMIN	{"catatan":"Lengkapi tujuan pemakaian"}	200 OK, keputusan revisi	409 jika status tidak bisa direvisi.

Access API - dikerjakan Tadzkiroh

Method	Endpoint	Role	Body	Expected	Jika gagal
GET	{{baseUrl}}/api/access/ready	SATPAM	Tidak ada body	200 OK, list reservasi approved yang siap akses	Jika kosong: pastikan ada approval setuju.
POST	{{baseUrl}}/api/access/check-in	SATPAM	{"reservationId":"RSV-001"}	200 OK, checkInTime terisi	409 jika belum approved atau sudah check-in.
POST	{{baseUrl}}/api/access/check-out	SATPAM	{"reservationId":"RSV-001"}	200 OK, checkOutTime terisi	409 jika belum check-in.
POST	{{baseUrl}}/api/access/issues	SATPAM	{"reservationId":"RSV-001","catatanPelanggaran":"Kunci terlambat dikembalikan"}	200 OK, catatanPelanggaran tersimpan	400 jika catatan kosong.

Notification API - dikerjakan Ajda

Method	Endpoint	Role	Body	Expected	Jika gagal
GET	{{baseUrl}}/api/notifications	Semua role login	Tidak ada body	200 OK, list notifikasi user login	Jika notifikasi user lain muncul: bug authorization.
PATCH	{{baseUrl}}/api/notifications/{notificationId}/read	Semua role login	Tidak ada body	200 OK, statusBaca=true	404 jika notificationId tidak ada.
POST	{{baseUrl}}/api/notifications/test	ADMIN/dev only	{"penerimald":"USR-001","penerimaRole":"MAHASISWA","pesan":"Tes notifikasi"}	201 Created, notifikasi baru	Hapus endpoint test sebelum demo jika tidak diperlukan.

6. Starter code per anggota

Snippet ini hanya starter gambaran. Jangan copy semua sekaligus. Isi file sesuai urutan timeline dan jobdesk.

Amelia - User/Mahasiswa

```
@Entity
public abstract class User {
    // TODO field: id, nama, email, noHp, passwordHash
    // TODO behaviour: login, logout, ubahProfil
}

@Entity
public class Mahasiswa extends User {
    // TODO field: nim, prodi, angkatan
    // TODO behaviour: ajukanReservasi, batalkanReservasi, lihatStatusReservasi
}
```

Ali - Room/ReservationService

```
@Entity
public class Room {
    // TODO field: roomId, namaRuang, gedung, kapasitas, statusAktif
    // TODO behaviour: aktifkan, nonaktifkan, ubahStatusAktif, getInfoRuang
}

@Service
public class ReservationService {
    // TODO createReservation, validateAvailability, cancelReservation, getReservationHistory
}
```

'Arif - Reservation/Approval

```
@Entity
public class Reservation {
    // TODO field: reservationId, mahasiswa, room, tanggal, jamMulai, jamSelesai
    // TODO field: tujuan, status, createdAt, cancelledAt, approval, accessRecord
    // TODO behaviour: ajukan, ubahStatus, batalkan, validasiWaktu, isCanBeCancelled
}

@Entity
public class Approval {
    // TODO field: approvalId, reservation, admin, keputusan, catatan, reviewedAt
    // TODO behaviour: setuju, tolak, mintaRevisi
}
```

Atha - Admin/Notifiable

```
public interface Notifiable {
    // TODO receiveNotification
}

@Entity
public class Admin extends User implements Notifiable {
    // TODO field: unitKerja
    // TODO behaviour: verifikasiReservasi, setujuReservasi, tolakReservasi, mintaRevisiData
}
```

Tadzkiroh - Satpam/AccessRecord

```
@Entity
public class Satpam extends User implements Notifiable {
    // TODO field: shift, posJaga
    // TODO behaviour: konfirmasiCheckIn, konfirmasiCheckOut, catatKendala
}

@Entity
public class AccessRecord {
    // TODO field: recordId, reservation, satpam, checkInTime, checkOutTime, catatanPelanggaran
    // TODO behaviour: checkIn, checkOut, laporkanKendala
}
```

Ajda - Notification

```
@Entity
public class Notification {
    // TODO field: notificationId, penerima, pesan, statusBaca, createdAt
    // TODO behaviour: kirim, tandaiDibaca
}

@Service
public class NotificationService {
    // TODO sendNotification, sendStatusUpdate
}
```

7. Skenario bagaimana jika gagal

Kendala	Penyebab	Pencegahan	Penanganan
Konflik saat Sync/Pull	Dua orang mengedit file sama.	Lock file di grup sebelum edit. Stage hanya file sendiri.	Jangan klik asal. Buka Merge Editor VS Code, hubungi pemilik file, pilih bagian yang benar, lalu test ulang.
Main rusak setelah push	Ada error compile atau app tidak start.	Setiap orang wajib run test sebelum Sync.	Koordinator minta stop push. Orang terakhir memperbaiki atau revert commit kecil lewat VS Code/GitHub.
Database tidak muncul di phpMyAdmin	MySQL belum jalan atau URL salah.	Jalankan XAMPP/MariaDB dulu. Pakai database reservasi_ruang.	Cek application.properties, username root, password kosong, port 3306.
API 401/403 di Postman	Belum login, role salah, atau CSRF token lama.	Login sesuai role dan ambil CSRF dari GET /login.	Ulang GET /login, POST /login, lalu test endpoint dengan session cookie yang sama.
API 400	Body JSON salah atau field wajib kosong.	Ikuti contoh body di dokumen ini.	Cek nama field, tipe data tanggal/jam, dan validasi di console.
API 409	Data valid tapi melanggar aturan bisnis.	Pahami aturan status dan jadwal.	Ganti jadwal, status, atau data test. Jangan anggap 409 selalu bug.
Anggota belum selesai	Satu modul menahan modul lain.	Kerjakan urutan dependency: User/Room dulu, lalu Reservation, Approval, Access, Notification.	Buat stub sementara berupa return kosong hanya jika tim sepakat dan tidak merusak test.
Tidak sengaja edit file anggota lain	Auto-format atau salah buka file.	Matikan format seluruh project. Edit file sendiri saja.	Sebelum commit, di Source Control unstage file yang bukan milikmu.
Postman endpoint 404	Endpoint belum dibuat atau URL beda.	Catat endpoint di grup sebelum coding.	Cek mapping controller, base path /api, dan path variable.
Demo mepet	Fitur belum semua stabil.	Freeze fitur sebelum demo.	Demo alur yang sudah stabil: login, room, reservation, approval, access, notification minimal.

8. Checklist sebelum demo

- Semua anggota sudah Pull terbaru.
- Tidak ada file owner yang diedit orang lain tanpa izin.
- Aplikasi start tanpa error.
- Login mhs, adm, stm berhasil.
- Postman flow jalan: room, reservation, approval, access, notification.
- Database berisi data demo yang cukup.
- Tidak ada push fitur baru setelah freeze.
- README berisi cara run yang benar.